

1

0:0:0-->0:0:21

Everyone. Today we're going to go over. In particular, we're going to go over engram language models. Have any of you heard of a language model before? I'm sure you've you've heard of large language models like chat, gpt, or claude,

2

0:0:24-->0:0:46

or gemini. And these are all large language models. However, looking at or taking away the whole large component, right? At the end of the day, these are regular you're not regular, but they're language models.

3

0:0:47-->0:1:4

These are all examples of more advanced or complex language models. In particular, generative ai language models that are trained on really large data sets and have a really large vocabulary. And as we'll see, are

4

0:1:4-->0:1:15

very large distributions of sequences of n grams, which is how we define a language model, and therefore, are what we call large language models.

5

0:1:15-->0:1:20

so Today we're going to go over

6

0:1:21-->0:1:42

the way language models work, what is and what an engram language model is. By the way, by n gram, we really just mean a sequence of n tokens.

7

0:1:43-->0:2:1

and The reason why we call it an n gram or gram is because gram here refers to is really short for grammatical unit,

8

0:2:1-->0:2:16

which, as we discussed in a previous lecture, is effectively like a morpheme, a meaningful grammatical unit, which is like a morpheme, which is analogous to or hopefully a token. If we token I something meaningfully,

9

0:2:17-->0:2:30

soreally, n gram language models are just language models uh that collect sequences of n tokens. As we'll see, this is really a distribution of n grams or n tokens.

10

0:2:30-->0:2:38

Before I jump into this, the key thing I want us to understand is that

11

0:2:39-->0:2:59

an engram or sorry, an engram, a language model, at the end of the day, before I jump into

the outline, at the end of the day, a language model is a distribution, a probability distribution. It could be normal.

12
0:2:59-->0:3:28
It could be something exponential looking. It could have 2 kind of means here, but the point is needs to sum to one. It's some form of a distribution. These are distributions

13
0:3:31-->0:3:46
at the end of the statistical distributions. And it's through these distributions that we hope to model so called language in the way it's used. okayLet's jump into the outline. So first, we'll introduce

14
0:3:46-->0:3:59
n gram language models, what an engram is, language models, which I sort of just did, and what motivates their usage in the first place. umWe'll go into n grams like fundamentally and practically what they are.

15
0:3:59-->0:4:14
uhAnd then we'll introduce the chain rule and the markov assumption and how that enables us to use engram language models um for practical things, like predicting the next word without too much computational cost,

16
0:4:14-->0:4:29
maximum likelihood estimation, which will basically be a way of saying this is the most likely next sequence, given some input sequence, as you'll see, will include beijing to statistics here.

17
0:4:30-->0:4:43
andAs you might have guessed, these are all part of some larger statistical model, which means willing to train our model and test our model on test sets. Some measures of usefulness such as per perplexity.

18
0:4:44-->0:4:54
andThen some more practical things like sampling from language models themselves, the zero probability problem, smoothing techniques, which arewhich is important. Um

19
0:4:54-->0:4:55
Basically, how do you predict

20
0:4:56-->0:5:14
the a word that you haven't seen in, the thing you trained on or trained? How do you predict unseen words? You got a smooth uh and then things like entropy and cross entropy, which are measures of like disorientation of data okay jumping in.

21
0:5:15-->0:5:31

At the end of the day, the motivation behind large language models is just predicting the next word or the next sequence of words. Sometimes we'll make the synonymous with the next sequence of tokens. Here, a warm mental exercise is

22
0:5:31-->0:5:55
what word is likely to follow after this sequence of words? In other words, complete this sentence, the water of walden pond is so beautifully. whatWhat word do you think should come after beautifully? Should that be a noun?

23
0:5:56-->0:6:17
Should it be an adjective? What do you think? Should be there? Moreover, what do you think probably shouldn't be there? For example, a likely word that would come after that is blue, green or clear.

24
0:6:19-->0:6:38
These are colors or maybe lack thereof. And beautifully is like I think it's an adverb, but it's a way of modifying this noun or these nouns. They could also technically be adjectives, but I think I you know what I mean.

25
0:6:39-->0:6:55
The water walden pond is so beautifully blue or beautifully green. It would not likely be these other nouns like refrigerator or this this, by the way, isalso is what we call a determiner.

26
0:6:55-->0:7:0
So

27
0:7:0-->0:7:18
we kind of intuitively have some sense as to what word should come after, uhnot just one other word beautifully, but a sequence of words, keep in mind though that we if we didn't have this sequence of words, before this word here, we probably could have like a

28
0:7:18-->0:7:20
a sense as to what kind of word would come next. Right?

29
0:7:20-->0:7:34
In other words, let's say we did not have all of these words before, and I gave you the word beautifully. What would come after that word? If this was like part of a sentence?

30
0:7:35-->0:7:53
But I didn't tell you this part of the sentence. I only said this is one beautifully refrigerator, still probably makes no sense. Beautifully. This may be a little bit more sense, but not that much sense. Still, these words likely work, right?

31
0:7:54-->0:8:8

Eliminating all this stuff before this kind of just reduces the space of words or the the amount of words that it could be. But we have a sense still of what ought to come after this in terms of the types of words.

32

0:8:9-->0:8:28

soWe have some intuition. rightWe kind of get a feel for this. But how do we formalize that intuition? That's really what nlp is about. soTo formalize this language, uhthis intuition, which really just has to do with, like, what do I think will likely come after this?

33

0:8:29-->0:8:43

We introduced this concept of a language model, which is a in practice, it's a machine learning model. Really, as we'll see, it's just a distribution that predicts the next likely word um given a sequence of words.

34

0:8:44-->0:8:56

soIn essence, it assigns a probability to each possible next word. And the way it assigns that probability, umas I was alluding to before, is via a distribution.

35

0:8:58-->0:9:5

Language models can assign a probability to an entire sequence of words, as well. As we'll see, using uh the markov property.

36

0:9:5-->0:9:15

soFor example, let's say we have a language model that can tell us that this sequence has a high probability has a higher probability.

37

0:9:15-->0:9:33

Then a scrambled version of that sequence beneath that. rightWhat do we mean by this sequence? Has a higher probability than this sequence? Let's say, to come after another one. right wellIf we tokenize this, by words, let's call this t one, t two,

38

0:9:33-->0:9:56

t three, et cetera. Let's call this t and i'll say maybe this is like t one, umprime, t two prime, t one, t two, t three, et cetera. What i'm saying is the probability

39

0:9:57-->0:9:59

of this sequence,

40

0:10:2-->0:10:28

t one prime, t two prime, t three prime, is more likely to occur than t one, t two, t three, and llm we might have some intuition for that.

41

0:10:28-->0:10:29

Right

42

0:10:30-->0:10:45

When we say the probability of this, we really mean the probability of this occurring after, as we'll see, after, given some input to a model. right In other words, these are these are sequences or texts that could come next

43

0:10:54-->0:11:13

given a sequence. A language model will tell us this. and The way it tells us this is by constructing a distribution of these sequences. To each of these is some t some sequence, some t one

44

0:11:13-->0:11:34

prime, t two prime. Each of these refers to one of these sequences. And all it's saying is this one? This sequence is the most likely, fundamentally large language models are built by training

45

0:11:35-->0:11:52

them to predict the words. So that can basically was, we'll see it takes a large amount of words, usually from some corpus and puts in a science probabilities to sequences that it's counted. How do you think we count these sequences or extract them?

46

0:11:52-->0:12:14

Give you a hint. Rag x some practical applications include you know auto complete or like grammar corrections that we've probably seen on our phones. Right? Like you know if I were to text someone something, and I type, let me see if I can type something now.

47

0:12:15-->0:12:34

Hi, my name, I don't know if you can see it, but I probably can't, but as I type, hi my name, but I pause at the m already, there's like a lighter, shaded e space, is that it thinks is going to come next.

48

0:12:34-->0:12:51

I'll see if I can block out my own. ah I will do it, but try this on your phone type in an incomplete sequence and see if it wants to complete something that's using a language model, most likely.

49

0:12:51-->0:13:0

okay So that's

50

0:13:0-->0:13:14

at a high level language model, but this lecture was titled n-gram language model. so What is an n-gram? Our language model is a probability distribution, and the and the word n-gram modified that which means

51

0:13:14-->0:13:31

this is an n-gram type of distribution or a distribution of n grams. well What is an n-gram

an? Engram? Is a sequence of n words. so You could think of it. Honestly, as any literally any sequence of words, it could be a sentence.

52
0:13:31-->0:13:53
It could be a part of a sentence. it's A you can basically just make gram equivalent to token or word. If we token eyes at the word level, so it's just a sequence of these. How long is this sequence? N

53
0:13:54-->0:14:1
so a bi gram is or 2 gram also known as a bi gram is just a sequence of two words.

54
0:14:1-->0:14:12
The water water of I could also say um of the or the river

55
0:14:14-->0:14:34
as if it came from another text, a trigger a 3 gram or a trigram is just a sequence of three words, the water of water of walden and you can go on write a 4 gram. I don't know what that would be um in english, a quad gram.

56
0:14:34-->0:14:48
But a foreground would be for example, the water of walden, 1234.

57
0:14:50-->0:14:53
and A 5 gram would be at the water of walden is or something, right?

58
0:14:58-->0:15:14
For what it's worth or it is worth noting. When we say n gram, we mean a sequence of n words as well as a probabilistic model that estimates the n th word, given a sequence of n minus one words. Key thing to note,

59
0:15:16-->0:15:31
sowe'll use an engram to denote that one of these sequences or a model that predicts the next word in a sequence of length n to the n th word and the sequence n minus one.

60
0:15:37-->0:15:43
Some key notation, i'm gonna use w_i to represent a word or token.

61
0:15:43-->0:15:55
By the way, we're also using word synonymous with token here. um Just for this chapter, we'll use w subscript one to n

62
0:15:56-->0:16:10
to also mean a sequence of n words from position one to n so we're we'll index at one. We'll use w sue that w subscript less than n

63
0:16:10-->0:16:14
to mean all elements up to and including n minus one.

64
0:16:14-->0:16:29
so This is, in other words, so in other words, this is basically w one, w_n minus one. This is equal to w less than n

65
0:16:30-->0:16:46
and the probability of word, I is literally just the probability of that single word. The probability of w_n given a sequence one, a sequence of words, one through w_n minus one

66
0:16:46-->0:17:4
is the conditional probability of a single word, given some sequence up to right before that word. Other special symbols will use are this triangle bracket um sentence, beginning marker,

67
0:17:4-->0:17:31
and then ascend its end marker, so almost like html tags. If you ever heard of that, so the problem, and by the way, all of these at the end of the day, as we'll see, our distributions, all of these probabilities will give us

68
0:17:34-->0:17:46
values that sum to or a set of values that sum to one, or its constituents as in its constituents will sum to one.

69
0:17:46-->0:17:59
so Our goal is to compute, for example, the probability that blue comes after,

70
0:17:59-->0:18:16
the water of walden pond is so beautifully. What? so In simple terms, this is w_n this is w one, this is w two, right? Et cetera, up to w_n minus one.

71
0:18:16-->0:18:38
and What is n here? N is 123456789. N is nine. and so N minus one is eight equals w eight. so We want to predict the 9th word. and As a result, we're going to have to construct n

72
0:18:38-->0:18:45
a 9 gram language model, which again is just this distribution.

73
0:18:45-->0:19:0
so How do we do that? The naive approach, the naive beijing approach you can call, it is simply by counting the number of times we see this sequence.

74

0:19:1-->0:19:24

That's what this c means. Count c equals count the number of times that sequence occurs.

75

0:19:24-->0:19:33

soWe count the sequence up to like w

76

0:19:34-->0:19:48

um I think I said, like less than n and w_n and divided by the count of basically,

77

0:19:48-->0:19:50

w less than n

78

0:19:57-->0:20:12

and that'll give us. And this will always be less than this, because we're adding another word here. and soEvery sequence that this is that is counted here will also be counted here. soIn other words, this will always be less than one.

79

0:20:13-->0:20:28

The problem here is somewhat obvious. I think the entire web is not big enough for reliable counts of all full sentences. Also, language is very much fluid and creative. New sentences are constantly being invented.

80

0:20:28-->0:20:48

I mean if you've ever heard of shakespeare, that's like how he made his name, then he innovated the english language as a result. So we need better ways to do it than just this naive approach. One, sorry, one way to go about this is to

81

0:20:48-->0:20:53

look at the chain rule of probability, or to use the chain rule.

82

0:20:53-->0:21:7

soIn other words, if I want to predict, remember, this is the probability of w_n given w one

83

0:21:8-->0:21:35

up to w_n minus one, right so given what this means is probability of of some word given given these words. soIn other words, if I were to say, here's so here's some words, right?

84

0:21:35-->0:21:56

What do you think this? And this is what we'd call our evidence, right? What word do you think would come after that one? That's what this probability represents. Another way we can do that, right?

85

0:21:56-->0:21:58

Because without computational

86

0:21:59-->0:22:16

you know Exploding, because counting all that would be difficult, would be using conditional probabilities or other conditional probabilities. In other words, we could take we could find the probability of a certain sequence, right which is

87

0:22:16-->0:22:35

the same as even though this notation looks different, it's effectively the same as this. right It's the probability of basically this specific sequence occurring. Same thing as this effectively. Probability of that sequence occurring,

88

0:22:35-->0:22:47

we can say, is the probability of the first word times the probability of the second word time, given the the first word times the probability of the third word, given the first two words,

89

0:22:47-->0:23:2

all the way back until we get to the first word to the k minus one one word, or k equals one to n in other words, we can use the chain rule to decompose joint probabilities into conditional probabilities.

90

0:23:2-->0:23:4

However,

91

0:23:5-->0:23:9

we this is very difficult to compute for very long histories.

92

0:23:9-->0:23:26

so Instead of computing probabilities, given an entire history, we can approximate only the last few words using. Or sorry, we can approximate the next word using only the last few words.

93

0:23:26-->0:23:48

Remember, when I was talking about how we might not only need, or we may not need all this to figure out what comes next, you could still tell that these words are still good for this these words should come next.

94

0:23:48-->0:24:5

Over these words. In other words, the probability of this coming next is greater than the probability of this coming next without this. right If you see this, if we're just given this most recent word, sometimes known as the previous state, right

95

0:24:5-->0:24:25

we we can still estimate that this probability is greater than this probability, that the probability of that w_n is blue is greater than the probability of that. w_n

96
0:24:26-->0:24:54
is this. Right? We don't need all these words to know that this is true. We only need this word to know that this is true. So how do we formalize that intuition? The markov assumption, what do you mean by this? Well,

97
0:24:54-->0:24:57
basically, we only need the word before the one. We're trying to predict

98
0:24:57-->0:25:1
to predict it to predict what word comes comes next.

99
0:25:1-->0:25:14
Now. Really, in practice, you only need what it means is you only need the last few words. For example, trivial with the by gram model. You only use the preceding word, you knowbecause with the by gram, it's just two words.

100
0:25:15-->0:25:33
um soYou can predict the next word, just given the previous one. soThe example I was using before is right. We like the problem. In other words, this is like aa restatement of the markov assumption with a diagram model, or an example of it, at least the probability that blue comes after this sequence.

101
0:25:33-->0:25:37
Is this roughly the same as the probability that blue comes after the word beautifully?

102
0:25:37-->0:25:49
soIn other words, the markov assumption is that the probability of a word depends only on a fixed window of previous words more. Strictly speaking, it only depends on the word that comes before it. But

103
0:25:50-->0:26:10
with an nlp you make you know kind of these economic trade offs. The n gram approximation of this says another, soin other words, expanding this to beyond 9 grams, like as in this model, the probability of a word coming after some sequence is approximately

104
0:26:11-->0:26:22
the probability of that same word coming after some smaller sequence before it. andYou can just plug in n to see how this plays out mathematically.

105
0:26:22-->0:26:31
okay soHow do we use this? Though?

106
0:26:32-->0:26:49
We use this more often than not with this concept of a maximum likelihood estimation. soWe

to use maximum likelihood estimation, we first get the counts of a certain word or sequence in a corpus, right?

107

0:26:53-->0:27:8

Then we normalize the counts so that they sum to one. soThe key thing here is we're trying to add the key thing we're trying to get at is how do we estimate n gram probabilities? right how do we how do we getHow do we actually estimate this?

108

0:27:12-->0:27:28

One way to estimate this is through maximum likelihood estimation with a diagram. For example, this takes getting the count, basically, of that sequence, or of the of this word of this of the number of times. You see this word and a word coming after it that you're trying to predict,

109

0:27:33-->0:27:59

divided by the sum of the count of all of this this word, the previous word, and all other words that have come after it. so thisThis is the previous word. This is the word

110

0:28:0-->0:28:6

to predict or to get the probability of being next.

111

0:28:6-->0:28:25

And this is all other words that have that have come after,

112

0:28:26-->0:28:59

oopssorry, afterafter w_n minus one as you can see this kind of this basically simplifies to this those take a mini corpus, which will denote the start and end of uh with these s tags.

113

0:28:59-->0:29:2

Let's take a couple,

114

0:29:2-->0:29:23

excuse me, sentences here, right? A corpus is just something with a bunch of sentences. umOne sentence is I am sam the next sentence is, sam, I am. The next sentence is I do not like green eggs at ham. Let's compute the background probabilities. First,

115

0:29:24-->0:29:58

count the by grams. I'm going to pause and let everyone take a minute just to read this slide. All good. So what is, thei'm sure you're a little confused about what this means.

116

0:29:58-->0:29:59

Right?

117

0:30:0-->0:30:44

Si is two. What am I counting here? Can anyone guess? Next? thisThis weird s thing, sam is one. S is three. I count of I

118

0:30:45-->0:31:13

I appears 3 times across all these sentences. 3 times I am occurs. Once I do occurs. Once we compute these ratios, we're all set pretty simple, right?

119

0:31:18-->0:31:41

Some more examples. right these areWe're just computing the probabilities of by grams. Our general formula looks like this so for an n gram of size, capital n the probability of a word n coming after, a word w_n coming after some

120

0:31:41-->0:31:43

sequence of words, before that.

121

0:31:43-->0:32:4

One um is just the the ratio of those counts. This is also called a relative frequency. And this is a language model. soIt's called a relative frequency as we

122

0:32:4-->0:32:20

divide the observed frequency of a sequence by the observed frequency of its prefix. soIt's kind of like what we were doing before, but with a shorter with a naive approach. But with a shorter time frame, sowhy is this a maximum likelihood estimator?

123

0:32:20-->0:32:39

Because it maximizes the likelihood of the training set t this this corpus, given a model m so it maximizes, and we say it maximizes. We mean that it finds the argument,

124

0:32:40-->0:32:59

that isthe maximum value that gives us the maximum value of that probability distribution of this distribution. A real example would be the berkeley restaurant project.

125

0:32:59-->0:33:4

soI'm not gonna go over this in too much detail now,

126

0:33:4-->0:33:37

but we can let's look at the background counts of these sentences. soHere we have counts in tabular form of by grams for the for these eight words. So this is also known as a coincidence matrix. And what this tells you is

127

0:33:38-->0:33:58

how much how many times I has gone next to I eyes gone next to one, eyes gone next to two, eyes on next to eat, eyes, gone next to chinese and so forth, same thing.

128

0:33:58-->0:34:0

this is This tells you how many times want has gone.

129

0:34:0-->0:34:10

next to I want has gone next to one want has gone next to 21 has gone next to eat and so forth. so Cell i j

130

0:34:10-->0:34:29

I being some row, sorry, this I is refers to a row. J refers to a column, is the count of column, word, row following column, word, row, following row word. so It's the count of um

131

0:34:31-->0:34:56

this word following this word. Most of these values, though, as you'll see, are 012 unigram counts. We could do this, right? and So basically, what this will give you is the ratios are the numbers that to create ratios from before. So these will give you those cs.

132

0:34:56-->0:34:58

Those c one 2.

133

0:34:59-->0:35:17

Again, this is just for a by gram. so This coincidence matrix only works in the case of a by gram. For a unigram, we just need you know a vector to get by gram probabilities. Again, we just divide the by gram, count by its rose unigram, count

134

0:35:17-->0:35:31

so the formulas like so woo after a while you know we can compute these probabilities and so forth.

135

0:35:31-->0:35:46

And then we can compute sentence probabilities via conditional probabilities. And exercise would be try computing ps

136

0:35:46-->0:36:7

try computing basically the probability of this sentence occurring. so What do these probabilities capture? well Ideally, they capture linguistic phenomena encoded in by gram statistics, syntactic patterns, task specific patterns, and cultural and pragmatic patterns, so

137

0:36:7-->0:36:14

basically catches on to things statistically. Cool.

138

0:36:14-->0:36:29

Now we have a way of coming up with the next word of the next sequence, given a sequence. How do we evaluate? How do we know how good that model of the languages? There's 2 categories or types of evaluation.

139

0:36:29-->0:36:47

There's extrinsic valuation and intrinsic valuation evaluation, excuse me. There. So you can the extrinsic valuation evaluation is basically to measure how much like and a language model improves

140

0:36:47-->0:36:56

um based on its usage, whereas intrinsic evaluation measures the quality of independent application.

141

0:36:56-->0:37:9

umSo we'll go into that in detail with perplexity. uhLike any statistical model. We need a training set, a development set, and then a test set.

142

0:37:9-->0:37:22

Your training set is simply a corpus, right? Or data used to learn model parameters, umwhich for n grams, you knowit's just a corpus or which we can get counts of things, development set. Um

143

0:37:23-->0:37:39

If we use for tuning and testing during the development, its development can run many experiments. And it's again, this set is just like another corpus. And the test set is data we did not use in either of those. Tell that data for evaluation at which must not overlap with the training set.

144

0:37:39-->0:37:59

andWe only use once or very few time for like formal evaluation. We've probably, if you haven't taken a machine learning class, um you knowthere's a 8020 split, umbut there's also different ways to split data set,

145

0:37:59-->0:38:8

data sets into test development or tests, uh yeahdev, andand i'm sorry, training, dev and test.

146

0:38:8-->0:38:23

Key things to note or that your is that your test set must be large enough for statistical power. I recommend looking into what statistical power means.

147

0:38:23-->0:38:44

Basically, it refers to having an amount of data that will allow you to catch two to accurately catch false positives. And ideally, your training set will just have as much data as possible. Other things to consider

148

0:38:44-->0:39:1

um is data contamination. So make sure your test set is away from your training set properly. umThis can result in basically just uh weird behaviors in

149

0:39:1-->0:39:5

testing, as well as just huge inaccuracies and actual perplexity.

150

0:39:5-->0:39:15

So in other words, the measure our measures of a model will go kind of weird. It will be less meaningful if our training and tests overlap.

151

0:39:21-->0:39:41

Like anything. A better model just comes from better data and better best practices. um Intuitively, right a better model is better at predicting the next word, right? and It's less surprised by the test set, which means there's less

152

0:39:41-->0:39:52

there's less volatility or or variance in in um in predictions over time or over iterations.

153

0:39:54-->0:39:55

ooh Here, they

154

0:39:56-->0:40:12

the problem with using just raw probabilities, though, that is that probability depends on sequence length and longer. Text can kind of inherently create smaller probabilities faster, which means it's hard to compare across uh text lengths.

155

0:40:13-->0:40:32

A solution to this to normalize probabilities. Across lengths of text is this concept of perplexity perplexity. At the end of the day, it's just a formula. so The perplexity of some test set is a probability of that sequence

156

0:40:32-->0:40:37

to the power to the negative power of one over n or to the negative one over n th root.

157

0:40:37-->0:40:54

so In other words, this using the chain rule, we can say this, the key probability or property here is that uh the inverse probability

158

0:40:55-->0:41:30

is normalized by length so for unigram. Perplexity looks like this. For a by gram, it looks like this. we can We can expand this to any n gram from before. For example, the wall street journal, our training set, if we were to just train on words from like

159

0:41:30-->0:41:51

all wall street journal articles, right? There'd be 38 million words. We could test it on 1.5 million words and compare unigram and trigram models, as we would see, more context would lead to lower perplexity, which implies a better prediction.

160

0:41:51-->0:42:3

soThe one thing here to note is that the lower the perplexity, ideally, like the that implies a higher meaningful prediction. soIf you're getting better predictions, your perplexity should be lower.

161

0:42:14-->0:42:22

soSome critical requirements for measuring for measuring models via complexity, umsome critical requirements of that.

162

0:42:22-->0:42:39

There's no test set knowledge. In other words, language models must be built without any knowledge of a test set. Right? So in other words, you're testing training, so just need to be separate. And the vocabulary is in your training and tests need to um need to be the same or roughly the same.

163

0:42:41-->0:42:48

Perplexity in terms of task performance. umIntrinsic improvement does not mean extrinsic improvement.

164

0:42:48-->0:42:56

So in other words, just because it has good perplexity, doesn't mean people are going to really uh people are going to

165

0:42:56-->0:43:17

know exactly or feel good about its responses, right? umJust because, in other words, just because data scientists say a model is good, doesn't mean if I use it, i'm gonna say it's good. um but itBut it usually correlate with task improvements. At the ideally, right? You do both.

166

0:43:23-->0:43:33

Perplexity can also be described in terms of a weighted branching factor. So a branching factor is the number number of possible.

167

0:43:33-->0:43:40

Next words that can follow any word. For example, a unigram is a uniform color language

168

0:43:40-->0:43:57

that consists of the words, red, blue, green, where each word is equally likely, by the way, a language is just a set of words, simple as that. So let's say we have a language that just consists of the words, red, blue, green. umEach word is equally likely. One third, the test set.

169

0:43:57-->0:44:21

Us says red red red blu, the perplexity of this sett, if we use the formula from before, is one over three, again, this is minus one over n or to the one minus one over n to plexer t

end up being freed.

170

0:44:24-->0:44:42

The interpretation here is that so given that, we have remember how a language model is, just a probability of words, right a probability of sequences of words, well what if we have a uniform distribution? right What if this is our language model?

171

0:44:42-->0:44:44

Everything is just

172

0:44:47-->0:44:54

1/3. In this case, the perplexity is the branching factor for

173

0:44:54-->0:44:57

this distribution and the uniform distribution.

174

0:44:58-->0:45:27

What do you mean by that? We mean that this are the number of possible next words that could follow any word, right? There's only three words. Any word is equally likely to come after the next one. So expanding from that, right?

175

0:45:27-->0:45:47

Let's say our language model is not just some uniform distribution. um Let's say there's different probabilities of of the of of this unigram occurring. Remember this this.

176

0:45:47-->0:46:9

This is still a language model, right because this adds up to 10.80.1 is 0.1 that's that's up to one. and Let's say we have some corpus right that says red red red, blue. The perplexity of this is 1.89.

177

0:46:11-->0:46:30

so There's still three possible next words with a branching factor of three. But perplexity is lower because the distribution is skewed, you know obviously, towards red, which is good, right? That means we have a we have a higher means our model is better because it knows what

178

0:46:30-->0:46:48

to predict more. Before with 1/3, it could be anything equally, right? And it's not a good model, because just rolling dice, here, we could likely say the next word is gonna be red. That's why and that makes our model better. Thus our perplexity is lower or ought to be lower.

179

0:46:50-->0:46:53

As a result, we can interpret our perplexity as a weighted branching factor.

180

0:46:53-->0:47:12

soHow do we visualize language models? Before ii drew a distribution? rightBecause I want us to see what it looked like.

181

0:47:15-->0:47:38

andMoreover, I drew it because I want us to really grasp that at the end of the day, a language model is just a distribution. It looks something like this, which means we can sample from it. So we can use a language model as something we can sample from

182

0:47:38-->0:47:59

by simply choosing random sentences according to the likelihood under our model. More probable sentences are obviously going to be generated more more often and vice versa. umThe concept of sampling, as well as the concept of entropy, which we'll introduce in a sec,

183

0:47:59-->0:48:13

was introduced by shannon and miller and selfridge in the 40 S in the in 48 and 50. umAnd when you sample a language model, you kind of get a feel for what the model has learned, right?

184

0:48:13-->0:48:31

It's just just by saying, hey, whatwhat's most likely to come based on that come after? uhIf I were to just take something out of it, right? There's tons of different sampling algorithms, a unit gram sampling process.

185

0:48:31-->0:48:36

So for example,

186

0:48:36-->0:48:57

if the probability of v is 0.06, then v occupies this interval right $0 \sim 0.6$ on that distribution. What do we mean by this interval? we meanBy an interval? andI mean it may not always be this exact interval.

187

0:48:57-->0:49:29

There will be an interval 0 to 0.6 a by gram sampling process.

188

0:49:29-->0:49:36

okay wellWhat if we want to sample? Like, see what aa language model would give us? If we asked it for two words,

189

0:49:37-->0:49:54

you could just generate a random diagram beginning with some sentence. Let w equal the second word of that sentence and generate the program starting with that word. Repeat with new words until you generate you know a sentence, basically. And you can extend this right to higher order n grams.

190

0:49:56-->0:49:57

Um

191

0:49:58-->0:50:14

If you do this with a corpus, right? you can kind of Like you can use this to do things like predict a new shakespeare play. what would You could say, what would shakespeare say? If we gave him this word to start with what would come next, and then we'll come after that word.

192

0:50:15-->0:50:27

If we train to model a language model on all of shakespeare, there's a paper. There's a website out there called random math, paper generator that uses like markov chains.

193

0:50:27-->0:50:32

and And basically this to predict um uh

194

0:50:33-->0:50:49

to to give you randomly generated math papers that look completely absurd. But once in a blue moon, they kind of make sense. And um it's it's silly, but it demonstrates this concept of being able to literally predict the whole math paper,

195

0:50:49-->0:51:8

which can sometimes what you mean. it's It sounds absurd, but it can sometimes make sense. I think it's called random map, random math paper, generator dot com or something. Sure. If you google random math paper generator, it'll come up and you can type in your name

196

0:51:8-->0:51:15

or or select a bunch of famous mathematicians names and it'll generate a paper with your name or and and or those mathematicians.

197

0:51:15-->0:51:24

And uh it'll say and it'll have a funny title like it's something completely abstract like you know the 4th

198

0:51:24-->0:51:42

um 4th order cohomology, funk doors uh omit topological, invariance, isomorphic to the cone, you know something silly. But if you read it, there's crazy math. Anyways.

199

0:51:44-->0:52:4

so One problem that we get when training on any finite corpus is that we'll miss some acceptable sequences. In other words, and this is what this kind of leads into is the fact that um you will basically you're you're going to,

200

0:52:5-->0:52:27

you're if you want to predict new things, you need to train. you you You need to either have them in your training set or you're doomed, given what we've already done. Right? In other

words, we can't predict words, we haven't been trained on. One way to

201

0:52:29-->0:52:42

to fix this is this concept of smoothing or discounting, which shaves off probability mass from frequent events and ensures all n grams have zero probability. I'm sorry, I have non zero probability.

202

0:52:45-->0:52:57

Smoothing techniques include laplace smoothing, add k smoothing interpolation, and stupid back off modern neural language, like lms have different approaches to this, though.

203

0:52:57-->0:53:3

So all we're doing here is figuring out little tricks to make sure that like

204

0:53:3-->0:53:9

we never divide by zero in our counts in the denominators that we were talking about before.

205

0:53:9-->0:53:23

soAdd one smoothing. All this means is add one to all engram counts before normalizing, basically just make sure that we're not dividing by zero. Right? Laplace smoothing

206

0:53:25-->0:53:40

refers to, let's say, with our original maximum likelihood estimation, adding one of the numerator, and adding the book besides the vocabulary to the denominator.

207

0:53:40-->0:53:45

soEvery count that we do, we just add one,

208

0:53:46-->0:54:15

except for the in the denominator. we haveWe add the vocabulary for by, and that's for a unigram for a by gram. It looks something like this for. Let's take like a corpus, right? We have a bi gram. We'll use like a coincidence matrix to to show you uh

209

0:54:15-->0:54:18

what words have come after which, right?

210

0:54:18-->0:54:31

soNow that the all those zeros we saw before are now ones. soEverything's going to have a probability. At some point, it's just going to be small relative to other things, and it will still add up to zero.

211

0:54:31-->0:54:56

You're sorry. And the probably the distribution the language model will still add up to one so with smooth problem. Smooth, say, excuse me, smoothed probabilities are coincidence

matrix.

212

0:54:57-->0:54:59

andThis is just a simple way to

213

0:54:59-->0:55:18

organize uh unigrams, like it's not universal, right? If you do this with 4 grams, it would look like a tensor like a cube thing. umBut the point is now we have real probabilities. You can deal with with the unigram. The problem, withthe with laplace smoothing

214

0:55:18-->0:55:34

is that you know we have we we're basically restructuring accounts. soWe can compute an adjusted count c that would give smooth probability. rightBut then there's too much probability mass moved to zeros. right soWe're literally artificially putting probabilities in places they don't need to be.

215

0:55:35-->0:55:47

um andIn practice, this actually ends up being poor one way to one other technique that avoids that problem is add k smoothing.

216

0:55:47-->0:55:52

soInstead of adding one, we add a fractional count, k so0.50.01.

217

0:55:53-->0:56:18

We just need to choose k um so again, this is just like a technique. We use to make sure that things don't we don't divide by zero. The idea is, or sorry, then there's another technique called interpolation.

218

0:56:20-->0:56:39

andThe idea with interpolation is using this engram hierarchy. soIn other words, when we have zero counts, we we tried a different n gram. so we weWe mix probabilities from different engram orders.

219

0:56:39-->0:56:55

We combine unigram, bi gram and trigram, probably like with weights. soA linear interpolation so interpolation just refers to like interpolating probabilities across the distribution, across n gram distributions.

220

0:56:58-->0:57:15

soA simple linear interpolation says, like we can sum these, uhsays the probability of word n coming after um these two words is equal to some interpolated linear interpolation of

221

0:57:15-->0:57:23

the probability of um of this word n plus the probability of word n coming after.

222

0:57:23-->0:57:30

It's basically the probability of this unigram, probability of this bi gram, and the probability of this uh trigram,

223

0:57:36-->0:57:56

where each of these, where this is a weighted average, where each of these landers was sums up to one. ifYou're sorry. If you sum land, does they sum to one? soLambda is depend on context. Again, it's just another trick. How do we set these values? Um

224

0:57:56-->0:58:9

You can use machine learning. There's a couple different techniques. umAt the end of the day. These are hyper parameters. So you have to set them somehow. Honestly, a lot of people, if they even use this, they set them like equally. And then let the machine let the let machine learning take its,

225

0:58:9-->0:58:26

you do hyper parameter tuning. um andThen there's lastly lastly, there's this idea of stupid back off, which ifwhich means if an n gram has zero counts, you go backwards to the n minus one gram, and then you continue going that way until you get. Um

226

0:58:27-->0:58:29

Do you find one with nonzero counts?

227

0:58:29-->0:58:51

Again, the problem, the reason why it's stupid is because it doesn't really produce true probability distributions. we're justWe're just kind of rearranging things. It's a technique, though, the formula for this is uh basically just

228

0:58:51-->0:59:11

search. umJust, yeah itit's kind of, as it says, here, like continue backing off until you find a non zero account. umFor this denominator. Some advantages are of this so that it's simple and efficient.

229

0:59:12-->0:59:18

um andThere's no complex parameter tuning. You're literally just making it like finding a den gram that works.

230

0:59:18-->0:59:31

So why why am I talking about perplexity and all this stuff in the first place? Like, why inverse probability?

231

0:59:31-->0:59:47

whyWhy are we taking the nth root in perplexity? Where does this formula come from? uh wellAll this stuff? Comes from the concept of information theory, sothe concept of entropy and cross entropy, which you may have heard before in machine learning,

232

0:59:48-->0:59:52

umor deep learning. He's provided deeper understanding of language modeling.

233

0:59:57-->0:59:58

Entropy,

234

0:59:59-->1:0:15

um this is like some it's not the most important for all this, but it's worth knowing for any random variable x over some other set $\mathcal{X}$ of random variables. and That other stylized x is just the set of random variables $\mathcal{X}$ of possible random variables

235

1:0:15-->1:0:31

um with a probability function. uh Shannon entropy, we call it h of x is that negative sum of all the over across all the probabilities of the probability of that variant variable times a log base two of that probability.

236

1:0:32-->1:0:35

This expression measures information or uncertainty.

237

1:0:35-->1:0:50

So why are we measuring uncertainty? Well, uncertainty, in this case, in many cases, refers to information. If that makes sense, so high entropy

238

1:0:50-->1:1:22

means there's more information, whereas low entropy means there's less information. oh I'm sorry. you know What? I think I need to switch these, around hi uh sorry, switch. these This should be more, and there should be less. So high. Entropy

239

1:1:22-->1:1:58

means less information and low entropy means more information. Apologies for that. so I'm not gonna go over this in too much detail. um You can compute the entropy of a sequence using these formulas. Um

240

1:1:59-->1:2:26

But this is more or less just to show you like the history of perplexity. And then there's things called cross entropy, which I'm sure you've heard of before. Perplexity, here's the slide I really think is important. Everything between, but I introduce entropy to here is more or less like a

241

1:2:26-->1:2:39

mathematical history, not the most important um perplexity, in a way, is like two uh this like almost like cross entropy, but to the like two to the power of like cross entropy.

242

1:2:39-->1:2:45

If you think about it or two to the power of

243

1:2:45-->1:3:2

shannon cross entropy, so in other words, you can find the perplexity of some sequence saying, take find the two to the power of the shannon entropy of that sequence, which ends up being this.

244

1:3:2-->1:3:8

and so the point of this is to explain perplexities form. Like, why does it look like this?

245

1:3:8-->1:3:25

The first place, right? Basically, you can rewrite this as this. The key takeaways are that. Language models predict words and assign probabilities to sequences at the other day. When we say assign probabilities to sequences, we really just mean it's a distribution.

246

1:3:26-->1:3:41

A distribution of what? Of n grams. That's why we call these n gram language models, um which only require using the markov assumption, only require us to know a certain window of words, before a certain word that we're trying to predict,

247

1:3:41-->1:3:58

uh when we use we can use maximum likelihood estimation to make that prediction, um where the estimators are the language models of those probability distributions. I think we've gone over multiple times how to train them a model. right Uh

248

1:3:58-->1:4:16

In particular, all you have to know is the formula for perplexity. Just know that low perplexity means a better model. Perplexity is inverse probability normalized by the sequence length. um There's some practical techniques and general theory we've gone over. um I recommend reading over these slides

249

1:4:16-->1:4:23

to go over that to review that. okay Thank you.